OPERATING SYSTEMS AND EMBEDDED SYSTEMS

A Comparative Study of MLFQ, Rate Monotonic (RM), and Earliest Deadline First (EDF) Scheduling Algorithms

A University-Level Research Paper

Student Name	**Najla Davarpanah**
University	**Eshragh**
Course	**Embedded Systems**
Date	**22 October 2026**

Prepared in partial fulfilment of the course requirements. This document is formatted for A4 printing.

Abstract

The choice of a CPU scheduling policy determines how processor time is allocated among competing tasks, and different policies encode fundamentally different assumptions about workload behaviour. This paper compares three algorithms that represent distinct design philosophies: the Multi-Level Feedback Queue (MLFQ), Rate Monotonic (RM), and Earliest Deadline First (EDF). MLFQ is a feedback-driven, multi-queue policy used in general-purpose and interactive systems, whereas RM and EDF are real-time policies analysed for periodic task sets. The comparison examines priority assignment, preemption, interactive response, deadline guarantees, schedulability analysis, overhead, starvation, and application domains. The paper explains the classical RM utilisation bound $U \leq n(2^{1/n} - 1)$, emphasising that it is a sufficient but not necessary condition, and the EDF condition $U \leq 1$ for independent, preemptive periodic tasks with deadlines equal to periods on a single processor. A shared illustrative task set shows that RM can meet deadlines even when U exceeds the RM bound, while EDF provides a schedulability guarantee up to full utilisation under its assumptions. The paper concludes that no algorithm is universally superior: MLFQ is appropriate for interactive general-purpose workloads, RM for predictable low-overhead periodic control, and EDF for real-time systems that require higher processor utilisation.

Keywords: CPU scheduling; MLFQ; rate monotonic; earliest deadline first; real-time systems; schedulability analysis.

Contents

1. Abstract
2. Introduction
3. Overview of the Three Algorithms
4. Technical Operation and Examples
5. Comparative Analysis
6. Advantages, Limitations, and Use Cases
7. Conclusion
8. References

1. Introduction

CPU scheduling is a central function of an operating system: it decides which runnable task receives the processor, for how long, and in what order. The policy affects responsiveness, throughput, fairness, and—in real-time systems—whether deadlines are met. Different scheduling algorithms encode different assumptions about the workload, and comparing them requires attention to those assumptions rather than to a single performance number.

This paper examines three influential algorithms. The **Multi-Level Feedback Queue (MLFQ)** maintains several priority queues and adjusts a process's queue level according to observed behaviour; it is designed for general-purpose and interactive systems in which burst lengths are not known in advance [2], [3]. **Rate Monotonic (RM)** is a fixed-priority algorithm for periodic real-time tasks: a task with a shorter period receives higher priority, and schedulability can be tested analytically [1]. **Earliest Deadline First (EDF)** is a dynamic-priority algorithm that always selects the task with the earliest absolute deadline; under the classical assumptions it is optimal for preemptive uniprocessor scheduling [1], [4].

Comparing these algorithms is useful precisely because they are not interchangeable. MLFQ is typically used for general-purpose time-sharing and interactive workloads, while RM and EDF are analysed for periodic hard real-time task sets. Applying RM or EDF utilisation tests directly to MLFQ would be a category error: MLFQ does not assume periodic tasks, known worst-case execution times, or deadlines equal to periods, and it provides no hard deadline guarantee. The paper's objectives are therefore to explain the operation of each

algorithm, compare their properties under clearly stated assumptions, provide concise illustrative examples, and identify the workload conditions under which each is appropriate.

2. Overview of the Three Algorithms

2.1 Multi-Level Feedback Queue (MLFQ)

MLFQ uses multiple ready queues Q_0, Q_1, …, Q_{n-1}, where Q_0 has the highest priority. A new process enters Q_0. The scheduler always selects from the highest-priority non-empty queue and uses round-robin within that queue. If a process consumes its allotted CPU time at a level—typically one or more quanta—it is demoted to the next lower queue. If it blocks for I/O before exhausting its allotment, some implementations promote it; this is a policy choice, not a universal MLFQ rule [2]. To prevent starvation, a priority boost periodically moves all ready processes back to Q_0 [2].

The number of queues, the quantum at each level, the demotion threshold, and the boost interval are implementation parameters. An educational operating system, Kafka-OS, illustrates a concrete configuration: four levels with quanta of 2, 4, 8, and 16 ms, demotion after quantum exhaustion, promotion on I/O blocking, and a 500 ms priority boost [5]. MLFQ is not a hard real-time algorithm; its behaviour depends on the arrival pattern and on the chosen parameters, and it provides no worst-case deadline guarantee.

2.2 Rate Monotonic (RM)

RM is a fixed-priority algorithm for periodic tasks. Each task τ_i is characterised by a worst-case execution time C_i, a period P_i, and a relative deadline D_i. Under the classical Liu–Layland assumptions, tasks are independent, preemptive, periodically released on a single processor, and have deadlines equal to their periods; execution times are constant [1]. RM assigns higher priority to tasks with shorter periods. For n independent periodic tasks, if the total utilisation $U = \Sigma\, C_i/P_i$ satisfies

$$U \le n(2^{1/n} - 1),$$

then the task set is guaranteed to be schedulable by RM [1]. The bound is 1.0 for $n = 1$, approximately 0.828 for $n = 2$, approximately 0.780 for $n = 3$, and approaches $\ln 2 \approx 0.693$ as n increases. It is a *sufficient* condition, not a necessary one: a task set with utilisation above the bound may still be schedulable, and an exact test—such as response-time analysis at the critical instant—is required to decide [1], [4]. RM is optimal among fixed-priority algorithms for implicit-deadline periodic tasks on a uniprocessor [1].

2.3 Earliest Deadline First (EDF)

EDF is a dynamic-priority algorithm. At each dispatch, it selects the ready task with the earliest absolute deadline. Priorities are therefore not fixed at task creation; they change as deadlines approach. Under the same classical assumptions—independent, preemptive, periodic tasks on a single processor with relative deadlines equal to periods—EDF is optimal: if any algorithm can meet all deadlines, EDF can [1], [4]. Its schedulability condition is

$$U \le 1,$$

which is both necessary and sufficient for implicit-deadline task sets under those assumptions [1], [4]. When relative deadlines are shorter than periods, a density condition based on $C_i/\min(D_i, P_i)$ provides a sufficient test, but exact feasibility requires a more detailed processor-demand analysis [4].

2.4 Scope and Domain of Comparison

MLFQ, RM, and EDF are not three alternative solutions to the same formal problem. MLFQ addresses general-purpose scheduling without prior knowledge of job lengths; RM and EDF address periodic real-time

scheduling with known parameters and explicit deadlines. The comparison in this paper therefore focuses on design philosophy, operational behaviour, and the conditions under which each algorithm's analytical guarantees apply.

3. Technical Operation and Examples

3.1 MLFQ Operation and Example

At each scheduling decision, MLFQ scans the queues from highest to lowest priority and selects the head of the first non-empty queue. Within a queue, processes are served in FIFO order; a preempted process returns to the tail of its current queue. A process is demoted when it exhausts its allotment at a level, and a periodic boost returns all ready processes to Q_0 [2]. The following illustrative example uses two queues: Q_0 with a 4 ms quantum and Q_1 with an 8 ms quantum. Processes A, B, and C arrive at time 0 with CPU bursts of 3, 10, and 6 ms respectively.

Table 1. Illustrative MLFQ scheduling trace (Q_0 quantum = 4 ms; Q_1 quantum = 8 ms).

Process	Arrival (ms)	Burst (ms)	Queue transitions	Completion (ms)
A	0	3	Q_0 → completes	3
B	0	10	Q_0 (4 ms) → Q_1 (6 ms)	17
C	0	6	Q_0 (4 ms) → Q_1 (2 ms)	19

The timeline is: A runs 0–3 and completes; B runs 3–7 and is demoted; C runs 7–11 and is demoted; B resumes in Q_1 and completes at 17; C completes at 19. A short CPU burst is served at high priority, while the longer CPU-bound processes are demoted. This is the intended feedback behaviour. The example is illustrative and is not a measurement from a real system.

3.2 RM and EDF: A Shared Illustrative Example

To compare RM and EDF fairly, both algorithms are applied to the same periodic task set under the classical assumptions: independent tasks, preemptive scheduling, a single processor, synchronous release, and deadlines equal to periods. Table 2 lists the task parameters.

Table 2. Shared periodic task set for the RM and EDF example (times in milliseconds).

Task	C_i	P_i	D_i	C_i/P_i
T1	2	5	5	0.40
T2	3	10	10	0.30
T3	4	20	20	0.20
Total utilisation U				**0.90**

The RM utilisation bound for $n = 3$ is $3(2^{1/3} - 1) \approx 0.7798$. Since $U = 0.90$ exceeds this bound, the bound does not guarantee RM schedulability. Nevertheless, an exact schedule over the hyperperiod (20 ms) meets all deadlines:

RM sequence: T1 (0–2), T2 (2–5), T1 (5–7), T3 (7–10), T1 (10–12), T2 (12–15), T1 (15–17), T3 (17–18). All jobs meet their deadlines: T3 completes at 18 ≤ 20, T2 at 15 ≤ 20, and T1 at 17 ≤ 20. This illustrates that the RM bound is sufficient but not necessary.

EDF is guaranteed to be schedulable because $U = 0.90 \leq 1$. One valid EDF sequence is: T1 (0–2), T2 (2–5), T1 (5–7), T3 (7–10), T1 (10–12), T2 (12–15), T3 (15–16), T1 (16–18). The difference from RM arises from EDF's dynamic deadline comparison: at time 15, T3 and T1 both have deadline 20, and either order meets

the deadlines. The example shows that EDF's utilisation test provides a stronger guarantee, while RM's bound can be pessimistic. It does not show that RM is always inferior; RM may be preferable when its lower runtime overhead and predictable fixed-priority behaviour are important [4].

4. Comparative Analysis

Table 3 compares the algorithms across the criteria that matter most in practice. The comparison is analytical and reflects the assumptions stated in Sections 2 and 3.

Table 3. Comparative properties of MLFQ, RM, and EDF.

Criterion	MLFQ	Rate Monotonic (RM)	Earliest Deadline First (EDF)
Scheduling policy	Multiple feedback queues; highest-priority non-empty queue; round-robin within a level.	Fixed-priority; shorter period = higher priority.	Dynamic priority; earliest absolute deadline = highest priority.
Priority assignment	Dynamic, based on observed CPU/I/O behaviour.	Static, assigned once from periods.	Dynamic, recomputed as deadlines approach.
Preemption	Preemptive (quantum expiry, higher-priority arrival, boost).	Preemptive under the classical model.	Preemptive under the classical model.
Main objective	Responsiveness and short-job turnaround without prior knowledge of burst lengths.	Guarantee deadlines for periodic tasks with low overhead.	Maximise utilisation while meeting deadlines; optimal for implicit-deadline uniprocessor tasks.
Interactive response	Good; interactive and I/O-bound jobs tend to stay at high priority.	Not a design objective.	Not a design objective.
Real-time deadline guarantees	None in the hard real-time sense.	Guarantees if the task set passes the utilisation bound or an exact test.	Guarantees if $U \le 1$ under implicit-deadline assumptions.
Schedulability analysis	No global utilisation bound; behaviour depends on configuration and arrival pattern.	$U \le n(2^{1/n}-1)$ is sufficient; exact test via response-time analysis.	$U \le 1$ is necessary and sufficient for implicit deadlines on a uniprocessor.
Overhead and complexity	Moderate to high: multiple queues, accounting, boosts.	Low: static priorities, simple dispatch.	Moderate: dynamic priority queue and deadline tracking.
Starvation and fairness	Starvation possible; mitigated by priority boosts or aging.	Low-priority tasks can starve under sustained overload; fixed priorities are predictable.	No starvation when $U \le 1$ and assumptions hold; overload can cause a domino effect.
Typical application	General-purpose OS, interactive time-sharing, mixed workloads.	Periodic embedded control, automotive, avionics, simple real-time systems.	Real-time systems needing higher utilisation or dynamic deadlines.

The table highlights three distinctions. First, MLFQ's priority is a *feedback* signal, whereas RM's priority is a static function of the period and EDF's priority is a dynamic function of the absolute deadline. Second, only RM and EDF come with utilisation-based schedulability tests, and those tests apply to periodic task models with explicit execution times and deadlines—not to MLFQ's general-purpose workload model. Third, the algorithms optimise different objectives: MLFQ favours responsiveness and short-job turnaround; RM favours predictable fixed-priority behaviour and low overhead; EDF favours high processor utilisation.

RM's utilisation bound becomes increasingly pessimistic as the number of tasks grows, approaching $\ln 2 \approx 0.693$. This does not mean that RM wastes 30% of the processor in every system; it means that the *sufficient* bound is conservative for large task sets, and exact analysis may admit higher utilisations [1], [4]. EDF removes this pessimism under its assumptions, but it introduces greater runtime overhead and less predictable behaviour under transient overload: a job that misses a deadline becomes the most urgent job, and one late-

ness can propagate to subsequent jobs—the so-called domino effect [4]. RM's fixed priorities make overload behaviour more predictable for the highest-priority tasks, which is valuable in safety-critical systems [4].

5. Advantages, Limitations, and Use Cases

MLFQ is attractive because it requires no advance knowledge of burst lengths, favours interactive and short jobs, and adapts to behaviour changes through feedback and boosts [2]. Its limitations follow from the same flexibility: performance depends on the number of queues, quantum sizes, demotion policy, and boost interval; starvation is possible without a boost; and the scheduler offers no hard deadline guarantee. A production implementation must also guard against gaming, such as a process issuing a small I/O operation just before its quantum expires to avoid demotion [5]. MLFQ is therefore best suited to general-purpose operating systems and interactive workloads, not to hard real-time control.

RM offers a simple, low-overhead fixed-priority policy with a well-understood schedulability test. It is optimal among fixed-priority algorithms for implicit-deadline periodic tasks on a uniprocessor, and its behaviour under overload is relatively predictable for high-priority tasks [1], [4]. Its main limitation is the pessimism of the utilisation bound for large task sets, which can lead system designers to under- utilise the processor if they rely only on the bound rather than on exact analysis. RM is also less suitable when deadlines differ substantially from periods or when tasks are aperiodic.

EDF achieves full processor utilisation under the implicit-deadline uniprocessor model and is optimal in the sense that it meets all deadlines whenever any algorithm can [1], [4]. This makes it attractive for systems that need to operate close to capacity. However, EDF requires dynamic priority management, which increases runtime overhead, and its behaviour under transient overload is less predictable than RM's: a missed deadline can cause a cascade of subsequent misses [4]. It is therefore often chosen when high utilisation is more important than predictable overload degradation, and when the system can be designed to handle occasional deadline misses gracefully.

In summary, the appropriate choice depends on the workload. MLFQ is a general-purpose policy for interactive systems; RM is a predictable fixed-priority policy for periodic control with modest utilisation; EDF is a high-utilisation dynamic-priority policy for real-time systems whose task parameters are known and whose overload behaviour can be managed. None of these algorithms is universally superior, and each relies on assumptions that must be checked before it is applied.

6. Conclusion

This paper compared MLFQ, RM, and EDF across priority assignment, preemption, response time, deadline guarantees, schedulability analysis, overhead, starvation, and application domains. MLFQ uses feedback across multiple queues to favour interactive and short jobs without prior knowledge of burst lengths, but it provides no hard real-time guarantee and cannot be validated with RM or EDF utilisation tests. RM assigns fixed priorities from task periods and guarantees schedulability when $U \le n(2^{1/n}-1)$, a sufficient but not necessary condition that approaches 69.3% for large task sets. EDF assigns priorities dynamically from absolute deadlines and, under the classical implicit-deadline uniprocessor assumptions, is schedulable if and only if $U \le 1$.

The illustrative example showed that a task set with $U = 0.90$ can be schedulable under RM even though it exceeds the RM bound, and is guaranteed schedulable under EDF. This demonstrates the difference between a conservative sufficient bound and an exact condition, and it reinforces the need to match the analysis to the algorithm's assumptions. MLFQ is most appropriate for general-purpose interactive systems, RM for predictable low-overhead periodic control, and EDF for real-time systems that require higher utilisation. The comparison is limited to the classical models described in this paper; blocking, shared resources, non-pre-

emptive sections, execution overhead, and multiprocessor scheduling can change the results and require additional analysis.

7. References

1. C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
2. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Chapter 8: Scheduling: The Multi-Level Feedback Queue. Madison, WI, USA: Arpaci-Dusseau Books. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf
3. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 9th ed. Hoboken, NJ, USA: John Wiley & Sons, 2014.
4. G. C. Buttazzo, “Rate monotonic vs. EDF: Judgment day,” *Real-Time Systems*, vol. 29, no. 1, 2005.
5. Kafka-OS, “MLFQ scheduling decision,” in *Kafka-OS documentation*, Zenodo, 2025. [Online]. Available: https://www.zenodo.org/records/19412718/files/KafkaOs.pdf